SAIL Coding Guide

Overview

This guide provides basic guidance and error avoidance for generating valid Appian SAIL interfaces using documented components and best practices. SAIL (Self-Assembling Interface Layer) is Appian's declarative UI framework for building responsive, accessible interfaces.

Core Principles

1. Interface Structure

- Use top-level layouts: `a!formLayout()`, `a!headerContentLayout()`, `a!paneLayout()`
- Nest layouts appropriately: `a!sectionLayout()`, `a!columnsLayout()`, `a!cardLayout()`
- Always include `contents` parameter for container components

2. Component Syntax

All SAIL components follow the pattern: `a!componentName(parameter: value)`

```
a!textField(
  label: "Customer Name",
  value: local!customerName,
  saveInto: local!customerName,
  required: true
)
```

Starting with local variables

IMPORTANT: All interfaces should start with the `a!localVariables` component. Any other functions or components references local variables have to be wrapped in this top-level component.

Local variable types

Local variables do not have a defined type. The type of a local variable is determined by the value of the variable at any given time.

```
a!localVariables(
  local!isActive: true,
  local!activeEmployees: a!queryEntity(
    entity: cons!EMPLOYEE_DSE,
    query: a!query(
      filter: a!queryFilter("active", "=", local!isActive),
      pagingInfo: a!pagingInfo(1, 10)
    )
  ).data,
  local!activeEmployees
)
```

In this example, local!isActive is of type Boolean and local!activeEmployees is of type Dictionary.

Variables without an initial value

Local variables that do not have an initial value are of type Null. This can often cause errors when trying to pass the local variable to a function, so you may need to cast it to the expected type.

```
a!localVariables(
  local!quantity,
  a!integerField(
    label: "Quantity",
    value: local!quantity,
    saveInto: local!quantity,
    validations: if(
      local!quantity<0,
      "Quantity must be greater than 0",
      ""
    )
  )
)
```

In this example, local!quantity<0 fails because the variable is not an integer. You can solve this by casting the variable before doing comparisons using the `tointeger()` function.

Updating a variable value

The type of a local variable can change if it is saved into from a component within an interface. The variable will now be of the same type as the new value that was saved into it.

```
a!localVariables(
  local!number: 1,
  a!floatingPointField(
    label: "Number",
    value: local!number,
    saveInto: local!number
  )
)
```

In this example, local!number starts out as an Integer. However, once a user interacts with the Number field, a Decimal value will be saved into local!number.

Test Data Guidelines

When creating code examples, use realistic but generic test data that helps users understand the component's purpose without being distracting or inappropriate.

Naming Conventions

✅ **Use generic, professional names:**

```
local!employees: {
  a!map(name: "Sarah Johnson", department: "Engineering", id: 1001),
  a!map(name: "Michael Chen", department: "Marketing", id: 1002),
  a!map(name: "Emily Rodriguez", department: "Sales", id: 1003)
}
```

❌ **Avoid:** - Real people's names (especially public figures) - Culturally insensitive or inappropriate names - Names that could be distracting or controversial

Data Values

✅ **Use realistic business data:**

```
local!orders: {
  a!map(orderNumber: "ORD-2024-001", amount: 1250.00, status: "Shipped"),
  a!map(orderNumber: "ORD-2024-002", amount: 875.50, status: "Processing"),
  a!map(orderNumber: "ORD-2024-003", amount: 2100.75, status: "Delivered")
}
```

✅ **Use placeholder patterns for sensitive data:**

```
local!customers: {
```

```
  a!map(email: "sarah.j@company.com", phone: "(555) 123-4567"),
  a!map(email: "michael.c@company.com", phone: "(555) 234-5678"),
  a!map(email: "emily.r@company.com", phone: "(555) 345-6789")
}
```

Quantities and Amounts

✅ **Use varied, realistic numbers:**

```
local!salesData: {45000, 52000, 48000, 61000, 58000}
local!inventory: {
  a!map(item: "Laptop", quantity: 25, price: 1299.99),
  a!map(item: "Monitor", quantity: 18, price: 349.99),
  a!map(item: "Keyboard", quantity: 42, price: 89.99)
}
```

❌ **Avoid:** - Sequential numbers (1, 2, 3, 4) - Round numbers only (100, 200, 300) - Unrealistic values ($999,999,999)

Dates and Times

✅ **Use relative dates with today():**

```
local!projects: {
  a!map(name: "Website Redesign", startDate: today() - 45, dueDate: today() +
30),
  a!map(name: "Mobile App", startDate: today() - 30, dueDate: today() + 75),
  a!map(name: "Database Migration", startDate: today() - 15, dueDate: today() +
45)
}
```

Status and Category Values

✅ **Use common business statuses:**

```
/* Order statuses */
"Pending", "Processing", "Shipped", "Delivered", "Cancelled"

/* Project statuses */
"Planning", "In Progress", "Review", "Complete", "On Hold"

/* Priority levels */
"Low", "Medium", "High", "Critical"

/* Departments */
```

```
"Engineering", "Marketing", "Sales", "HR", "Finance", "Operations"
```

Comments in Examples

✅ **Use helpful placeholder comments:**

```
a!startProcess(
  processModel: cons!PM_EMPLOYEE_ONBOARDING,
  processParameters: {
    employee: local!employeeData,
    department: local!selectedDepartment
  },
  onSuccess: {
    /* Add success handling logic here */
    a!save(local!showConfirmation, true)
  },
  onError: {
    /* Add error handling logic here */
    a!save(local!errorMessage, "Failed to start onboarding process")
  }
)
```

Essential Layout Components

Section Layout

Groups related content with optional collapsible functionality:

```
a!sectionLayout(
  label: "Customer Information",
  labelHeadingTag: "H2",
  contents: {
    /* Components go here */
  },
  isCollapsible: true,
  marginBelow: "STANDARD"
)
```

Columns Layout

Creates responsive multi-column layouts:

```
a!columnsLayout(
  columns: {
    a!columnLayout(
```

```
      width: "MEDIUM",
      contents: {
        /* Left column content */
      }
    ),
    a!columnLayout(
      width: "WIDE",
      contents: {
        /* Right column content */
      }
    )
  },
  stackWhen: {"PHONE", "TABLET_PORTRAIT"}
)
```

Use empty `a!columnLayout` components on either side of a column to center content.

Card Layout

Provides visual grouping with styling options:

```
a!cardLayout(
  contents: {
    /* Card content */
  },
  style: "STANDARD",
  padding: "STANDARD",
  showBorder: true,
  shape: "SEMI_ROUNDED"
)
```

Card Group Layout

Displays an arrangement of cards, with the same width and height. Prefer this approach any time there is a set of cards displayed together.

```
a!cardGroupLayout(
  labelPosition: "COLLAPSED",
  cards: {
    a!cardLayout(
      contents: {
        /* Card content */
      }
    ),
    a!cardLayout(
      contents: {
        /* Card content */
      }
```

```
    ),
    a!cardLayout(
      contents: {
        /* Card content */
      }
    ),
    a!cardLayout(
      contents: {
        /* Card content */
      }
    )
  },
  spacing: "STANDARD",
  cardWidth: "MEDIUM",
  cardHeight: "AUTO"
)
```

Common Input Components

Text Field

```
a!textField(
  label: "Email Address",
  labelPosition: "ABOVE",
  value: local!email,
  saveInto: local!email,
  required: true
)
```

Dropdown

```
a!dropdownField(
  label: "Department",
  choiceLabels: {"Sales", "Marketing", "Engineering", "Support"},
  choiceValues: {"sales", "marketing", "eng", "support"},
  value: local!department,
  saveInto: local!department,
  placeholder: "Select a department"
)
```

Radio Buttons

```
a!radioButtonField(
  label: "Priority Level",
  choiceLabels: {"High", "Medium", "Low"},
```

```
  choiceValues: {1, 2, 3},
  value: local!priority,
  saveInto: local!priority,
  choiceLayout: "STACKED"
)
```

Display Components

Rich Text Display

```
a!richTextDisplayField(
  labelPosition: "COLLAPSED",
  value: {
    a!richTextItem(text: "Status: ", style: "STRONG"),
    a!richTextItem(
      text: "Active",
      color: "STANDARD"
    )
  }
)
```

Tag Field

```
a!tagField(
  labelPosition: "COLLAPSED",
  tags: {
    a!tagItem(
      text: "Urgent",
      backgroundColor: "NEGATIVE"
    ),
    a!tagItem(
      text: "Customer Facing",
      backgroundColor: "ACCENT"
    )
  }
)
```

Action Components

Button

IMPORTANT: Button widgets should generally be placed inside a `a!buttonArrayLayout`, but there are exceptions: - `a!formLayout` uses `a!buttonLayout` for the `buttons` parameter -

`a!wizardLayout` takes an array of `a!buttonWidget` directly in the `primaryButtons` and `secondaryButtons` parameters

```
a!buttonArrayLayout(
  buttons: {
    a!buttonWidget(
      label: "Save Changes",
      style: "SOLID",
      color: "ACCENT",
      size: "STANDARD",
      value: true,
      loadingIndicator: true,
      saveInto: {
        /* Save actions here */
        /*a!save(local!submitted, save!value) */
      }
    )
  },
  align: "END"
)
```

Link

Use `a!richTextItem` instead of `a!linkField` for links.

```
a!richTextDisplayField(
  labelPosition: "COLLAPSED",
  value: {
    a!richTextItem(
      text: "Visit site",
      link: a!safeLink(uri: "http://www.appian.com"),
      linkStyle: "STANDALONE"
    )
  }
)
```

Link Style

IMPORTANT: Use `INLINE` for `linkStyle` ONLY when linked text is part of a sentence with other unlinked text. Otherwise use `STANDALONE`.

```
a!richTextDisplayField(
  labelPosition: "COLLAPSED",
  value: {
    a!richTextItem(
      text: "For more information, "
    ),
    a!richTextItem(
```

```
      text: "visit our website",
      link: a!safeLink(uri: "http://www.appian.com"),
      linkStyle: "STANDALONE"
    ),
    "."
  }
)
```

Data Display Components

Read-Only Grid

```
a!gridField(
  label: "Recent Orders",
  data: local!orders,
  columns: {
    a!gridColumn(
      label: "Order ID",
      sortField: "orderId",
      value: fv!row.orderId
    ),
    a!gridColumn(
      label: "Customer",
      sortField: "customerName",
      value: a!linkField(
        links: {
          a!recordLink(
            label: fv!row.customerName,
            recordType: recordType!Customer,
            identifier: fv!row.customerId
          )
        }
      )
    ),
    a!gridColumn(
      label: "Status",
      value: a!tagField(
        tags: a!tagItem(
          text: fv!row.status,
          backgroundColor: if(
            fv!row.status = "Complete",
            "POSITIVE",
            "SECONDARY"
          )
        )
      )
    )
  },
  showSearchBox: true,
  showRefreshButton: true
```

```
)
```

Chart Components

Column Chart

```
a!columnChartField(
  label: "Sales by Quarter",
  categories: {"Q1", "Q2", "Q3", "Q4"},
  series: {
    a!chartSeries(
      label: "2024",
      data: {45000, 52000, 48000, 61000}
    )
  },
  colorScheme: "RAINFOREST",
  showLegend: true,
  height: "MEDIUM"
)
```

Accessibility Best Practices

Heading Hierarchy

```
a!headingField(
  text: "Customer Dashboard",
  size: "LARGE",
  headingTag: "H1"
),
a!sectionLayout(
  label: "Account Information",
  labelHeadingTag: "H2",
  contents: {
    a!headingField(
      text: "Contact Details",
      size: "MEDIUM",
      headingTag: "H3"
    )
  }
)
```

Form Labels and Instructions

```
a!textField(
```

```
  label: "Social Security Number",
  instructions: "Enter your 9-digit SSN without dashes",
  value: local!ssn,
  saveInto: local!ssn,
  validationGroup: "personal_info",
  accessibilityText: "Social Security Number for tax purposes"
)
```

Common Parameter Patterns

Visibility Control

```
showWhen: a!isNotNullOrEmpty(local!selectedCustomer)
```

Validation Patterns

```
validations: {
  if(len(local!value) < 3, "Must be at least 3 characters", null),
  if(a!isNullOrEmpty(local!value), "This field is required", null)
}
```

Margin Control

```
marginAbove: "STANDARD",
marginBelow: "MORE"
```

Label Positioning

```
labelPosition: "ABOVE"  /* ABOVE, ADJACENT, COLLAPSED, JUSTIFIED */
```

Responsive Design

Columns Layout

Centering Single-Column Content

For single-column content, use a three-column layout with margin columns to prevent content from becoming too wide on large screens. The center column uses a fixed width on desktop but switches to `AUTO` on smaller screens for proper responsive behavior:

```
a!columnsLayout(
  columns: {
    a!columnLayout(
      width: "AUTO",
      showWhen: a!isPageWidth({"DESKTOP","DESKTOP_WIDE"})
    ), /* left margin - hidden on smaller screens */
    a!columnLayout(
      width: if(a!isPageWidth({"DESKTOP","DESKTOP_WIDE"}), "WIDE_PLUS", "AUTO"),
      contents: {
        /* All page content goes here */
      }
    ), /* main content - fixed width on desktop, AUTO on mobile/tablet */
    a!columnLayout(
      width: "AUTO",
      showWhen: a!isPageWidth({"DESKTOP","DESKTOP_WIDE"})
    ) /* right margin - hidden on smaller screens */
  }
)
```

Key Principles

- **Always include `AUTO` columns:** At least one column must have `width: "AUTO"` for fluid layout
- **Hide margin columns:** Use `showWhen: a!isPageWidth({"DESKTOP","DESKTOP_WIDE"})` to hide margins on mobile/tablet
- **Never use all fixed widths**: This causes responsive issues

Side by Side Layout

```
a!sideBySideLayout(
  items: {
    a!sideBySideItem(
      width: "MINIMIZE",
      item: a!imageField(
        labelPosition: "COLLAPSED",
        images: a!userImage(user: local!userId),
        size: "SMALL"
      )
    ),
    a!sideBySideItem(
      item: a!richTextDisplayField(
        labelPosition: "COLLAPSED",
        value: local!userName
      )
    )
  },
  alignVertical: "MIDDLE",
  stackWhen: {"PHONE"}
```

```
)
```

Interface Templates

Basic Form

```
a!formLayout(
  titleBar: a!headerTemplateSimple(title: "New Customer Registration"),
  contents: {
    a!sectionLayout(
      label: "Basic Information",
      contents: {
        a!columnsLayout(
          columns: {
            a!columnLayout(
              contents: {
                a!textField(
                  label: "First Name",
                  value: local!firstName,
                  saveInto: local!firstName,
                  required: true
                )
              }
            ),
            a!columnLayout(
              contents: {
                a!textField(
                  label: "Last Name",
                  value: local!lastName,
                  saveInto: local!lastName,
                  required: true
                )
              }
            )
          }
        )
      }
    )
  },
  buttons: a!buttonLayout(
    primaryButtons: {
      a!buttonWidget(
        label: "Submit",
        submit: true,
        style: "SOLID"
      )
    },
    secondaryButtons: {
      a!buttonWidget(
        label: "Cancel",
```

```
        value: true,
        style: "OUTLINE",
        saveInto: local!showCancelDialog
      )
    }
  )
)
```

Dashboard Layout

```
a!headerContentLayout(
  header: {
    a!cardLayout(
      contents: {
        a!richTextDisplayField(
          labelPosition: "COLLAPSED",
          value: {
            a!richTextItem(
              text: "Sales Dashboard",
              size: "LARGE",
              style: "STRONG"
            )
          }
        )
      },
      style: "ACCENT"
    )
  },
  contents: {
    a!columnsLayout(
      columns: {
        a!columnLayout(
          width: "NARROW",
          contents: {
            /* Sidebar content */
          }
        ),
        a!columnLayout(
          contents: {
            /* Main content area */
          }
        )
      }
    )
  }
)
```

Logical Operators

SAIL uses function-style logical operators rather than symbolic operators. Always use these functions when combining multiple conditions:

```
/* Correct usage of logical operators */
and(condition1, condition2, condition3)
or(condition1, condition2, condition3)
not(condition)

/* Incorrect usage - this will cause errors */
condition1 and condition2
condition1 or condition2
```

Examples:

```
/* Multiple conditions in showWhen */
showWhen: and(
  a!isNotNullOrEmpty(local!selectedItem),
  local!isEditable,
  local!hasPermission
)

/* Complex condition in if statement */
if(
  or(
    a!isNullOrEmpty(local!value),
    not(typename(typeof(local!value)) = "Number (Integer)"),
    tointeger(local!value) < 0
  ),
  "Please enter a valid positive number",
  null
)

/* Combining and/or conditions */
if(
  and(
    a!isNotNullOrEmpty(local!email),
    or(
      not(contains(local!email, ".com")),
      contains(local!email, "test")
    )
  ),
  "Please enter a valid email address",
  null
)
```

Remember that these functions can take any number of arguments, not just two:

```
/* Multiple conditions with and() */
and(condition1, condition2, condition3, condition4)
```

```
/* Multiple conditions with or() */
or(option1, option2, option3, option4, option5)
```

Null Checking and Default Values

To avoid null errors, use special functions to check whether values are (or are not) null or empty. Use default values as fallback.

Null Checking Functions

Checking that a local variable is not null or empty:

```
a!localVariables(
  local!basicInfo: "some content",
  a!sectionLayout(
    label: "Additional Info",
    contents: {},
    showWhen: a!isNotNullOrEmpty(local!basicInfo)
  )
)
```

Checking that a local variable is null or empty:

```
a!localVariables(
  local!basicInfo: null,
  a!messageBanner(
    icon: "info-circle",
    primaryText: "Enter the basic info to get started",
    showWhen: a!isNullOrEmpty(local!basicInfo)
  )
)
```

Default Values

```
a!localVariables(
  local!cookiePreference: null,
  a!textField(
    label: "Cookie Preference",
    value: a!defaultValue(local!cookiePreference, "Reject All"),
    readOnly: true
  )
)
```

Common Errors and How to Avoid Them

This section covers the most frequent SAIL syntax errors and provides examples of both incorrect and correct implementations.

1. Overuse of text() Function

❌ Incorrect - Unnecessary text() wrapping:

```
a!richTextItem(
  text: text("Status: Active"),  /* text() not needed for string literals */
  style: "STRONG"
)
```

✅ Correct - Direct string usage:

```
a!richTextItem(
  text: "Status: Active",  /* String literals don't need text() */
  style: "STRONG"
)
```

When to use text(): Only when converting non-text values to strings:

```
a!richTextItem(
  text: text(local!numericValue, `format`),  /* Note the required format
parameter: The output format string, supporting "date/time" format, "positive"
format, "positive;negative" format or "positive;negative;zero" format. */
  style: "STRONG"
)
```

2. Grid Component Validation Errors

❌ Incorrect - showSearchBox without Record Type data:

```
a!gridField(
  data: local!arrayData,  /* Array data source */
  showSearchBox: true,    /* Error: showSearchBox requires Record Type */
  columns: {
    /* columns */
  }
)
```

✅ Correct - showSearchBox with Record Type:

```
```

```
a!gridField(
  data: a!recordData(recordType!Employee),  /* Record Type data source */
  showSearchBox: true,                      /* Valid with Record Type */
  columns: {
    /* columns */
  }
)
```

✅ **Correct - Array data without showSearchBox:**

```
a!gridField(
  data: local!arrayData,  /* Array data source */
  /* showSearchBox omitted - not supported with arrays */
  columns: {
    /* columns */
  }
)
```

3. Layout Width Validation Errors

Side by Side Layout Widths

❌ **Incorrect - Invalid width values:**

```
a!sideBySideLayout(
  items: {
    a!sideBySideItem(
      width: "WIDE",  /* Invalid: Use AUTO, MINIMIZE, or 1X-10X */
      item: /* component */
    ),
    a!sideBySideItem(
      width: "NARROW",  /* Invalid: Use AUTO, MINIMIZE, or 1X-10X */
      item: /* component */
    )
  }
)
```

✅ **Correct - Valid width values:**

```
a!sideBySideLayout(
  items: {
    a!sideBySideItem(
      width: "AUTO",  /* Valid: AUTO, MINIMIZE, 1X-10X */
      item: /* component */
    ),
    a!sideBySideItem(
      width: "3X",  /* Valid: 1X through 10X */
      item: /* component */
```

```
    )
  }
)
```

Form and Wizard Layout Widths

This applies to both `a!formLayout` and `a!wizardLayout`

❌ Incorrect - Invalid contentsWidth:

```
a!formLayout(
  contentsWidth: "NARROW_PLUS",  /* Invalid width value */
  contents: {
    /* form contents */
  }
)
```

✅ Correct - Valid contentsWidth values:

```
a!formLayout(
  contentsWidth: "MEDIUM",  /* Valid: FULL, WIDE, MEDIUM, NARROW, EXTRA_NARROW */
  contents: {
    /* form contents */
  }
)
```

4. Component Confusion Errors

Milestone Components

❌ Incorrect - Using non-existent component:

```
a!milestoneStep(  /* This component doesn't exist (yet) */
  label: "Step 1",
  status: "COMPLETE"
)
```

✅ Correct - Using milestoneField with steps:

```
a!milestoneField(
  steps: {
    "Submit Customer Request",
    "Set Up On-Site Appt",
    "File Assessment",
    "Submit Proposal",
    "Submit Agreement",
    "Finalize Repairs"
```

```
  }
)
```

5. Invalid Parameter Values

Button Color Values

❌ Incorrect - Invalid color:

```
a!buttonArrayLayout(
  buttons: {
    a!buttonWidget(
      label: "Submit",
      color: "POSITIVE",
      /* Invalid: Use ACCENT, NEGATIVE, SECONDARY or any hex value from colors.md
*/
      style: "SOLID"
    )
  }
)
```

✅ Correct - Valid color values:

```
a!buttonArrayLayout(
  buttons: {
    a!buttonWidget(
      label: "Submit",
      color: "ACCENT",  /* Valid: ACCENT, NEGATIVE, SECONDARY, or any hex value
from colors.md */
      style: "SOLID"
    )
  }
)
```

Progress Bar Colors

❌ Incorrect - Invalid color:

```
a!progressBarField(
  percentage: 75,
  color: "SECONDARY"  /* Invalid: Use ACCENT, POSITIVE, NEGATIVE, WARN */
)
```

✅ Correct - Valid color values:

```
a!progressBarField(
  percentage: 75,
```

```
  color: "POSITIVE". /* Valid: ACCENT, POSITIVE, NEGATIVE, WARN, or any hex value
from colors.md */
)
```

Checkbox Value/Choice Mismatch

❌ Incorrect - Value not in choiceValues:

```
a!checkboxField(
  label: "Document Confirmation",
  choiceLabels: {"Confirmed"},
  choiceValues: {true},    /* choiceValues contains true */
  value: false             /* Error: false not in choiceValues */
)
```

✅ Correct - Matching values:

```
a!checkboxField(
  label: "Document Confirmation",
  choiceLabels: {"Confirmed"},
  choiceValues: {true},
  value: true              /* Value matches choiceValues */
)
```

Invalid Parameters

❌ Incorrect - Non-existent parameter:

```
a!textField(
  label: "Name",
  value: local!name,
  saveInto: local!name,
  skipAutoFocus: true  /* This parameter is not available externally */
)
```

✅ Correct - Valid parameters only:

```
a!textField(
  label: "Name",
  value: local!name,
  saveInto: local!name
  /* Use only documented parameters */
)
```

6. Component Availability Errors

Unavailable Components

❌ Incorrect - Using unavailable component:

```
a!dateRangeField(  /* This component is not available */
  startValue: local!startDate,
  endValue: local!endDate
)
```

✅ Correct - Use separate date fields:

```
a!columnsLayout(
  columns: {
    a!columnLayout(
      contents: {
        a!dateField(
          label: "Start Date",
          value: local!startDate,
          saveInto: local!startDate
        )
      }
    ),
    a!columnLayout(
      contents: {
        a!dateField(
          label: "End Date",
          value: local!endDate,
          saveInto: local!endDate
        )
      }
    )
  }
)
```

7. Layout Restriction Errors

Unsupported Component Nesting

❌ Incorrect - MultiColumnLayout in sideBySideLayout:

```
a!sideBySideLayout(
  items: {
    a!sideBySideItem(
      item: a!columnsLayout(  /* Error: columnsLayout not supported here */
        columns: {
          /* columns */
        }
      )
    )
```

```
  }
)
```

✅ **Correct - Use appropriate layout nesting:**

```
a!columnsLayout(
  columns: {
    a!columnLayout(
      contents: {
        a!sideBySideLayout(
          items: {
            a!sideBySideItem(
              item: /* single component only */
            )
          }
        )
      }
    )
  }
)
```

8. Redundant Elements Errors

Asterisks Added to Required Fields

❌ **Incorrect - Manually adding asterisks to required form fields:**

```
a!textField(
  label: "First Name *", /* Error: Asterisk added to label of required field */
  value: local!firstName,
  saveInto: local!firstName,
  required: true
)
```

✅ **Correct - Asterisk is added automatically when required is set to true:**

```
a!textField(
  label: "First Name", /* Correct: No additional asterisk */
  value: local!firstName,
  saveInto: local!firstName,
  required: true /* System adds asterisk automatically when required parameter is
set to true */
)
```

9. Invalid Chart Parameters

Pie Chart Legend Parameter

❌ **Incorrect - Using `showLegend` parameter for `a!pieChartField`:**

```
a!pieChartField(
  label: "Pie Chart",
  labelPosition: "COLLAPSED",
  series: {
    a!chartSeries(label: "Chart Series 1", data: 1),
    a!chartSeries(label: "Chart Series 2", data: 2),
    a!chartSeries(label: "Chart Series 3", data: 3)
  },
  colorScheme: "RAINFOREST",
  style: "DONUT",
  showLegend: true, /* Error: using showLegend parameter for pie chart is not
valid, even though it is used for other chart types */
  height: "MEDIUM",
)
```

✅ **Correct - Use `seriesLabelStyle: "LEGEND"` to show the legend for `a!pieChartField`:**

```
a!pieChartField(
  label: "Pie Chart",
  labelPosition: "COLLAPSED",
  series: {
    a!chartSeries(label: "Chart Series 1", data: 1),
    a!chartSeries(label: "Chart Series 2", data: 2),
    a!chartSeries(label: "Chart Series 3", data: 3)
  },
  colorScheme: "RAINFOREST",
  style: "DONUT",
  seriesLabelStyle: "LEGEND", /* Correct: pie chart uses a different syntax to
show the legend */
  height: "MEDIUM",
)
```

10. Button Style Inconsistency for `a!fileUploadField`

`a!fileUploadField` uses different button styles from other buttons

This is currently a bug in the system where this component uses the old button style names. IMPORTANT: This guidance is for `a!fileUploadField` and `a!signatureField only`!

❌ **Incorrect - Using `SOLID` and other correct button styles for `a!fileUploadField`:**

```
a!fileUploadField(
  label: "File Upload",
  labelPosition: "ABOVE",
```

```
  buttonStyle: "SOLID", /* Incorrect: Even though this is correct for all other
buttons, this causes an error with this component */
  saveInto: {},
  validations: {}
)
```

✅ **Correct - Use the old buttons style values with `a!fileUploadField`:**

```
a!fileUploadField(
  label: "File Upload",
  labelPosition: "ABOVE",
  buttonStyle: "STANDARD", /* Valid values: "PRIMARY", "SECONDARY" (default),
"STANDARD", "LINK" */
  saveInto: {},
  validations: {}
)
```

11. Valid icons

Use only icons that part of the supported set listed in `branding/icons`

❌ **Incorrect - Making assumptions about what icons are available:**

```
a!richTextDisplayField(
  labelPosition: "COLLAPSED",
  value: { a!richTextIcon(icon: "shield-alt") }
)
```

✅ **Correct - Check that icons are available in full reference list:**

```
a!richTextDisplayField(
  labelPosition: "COLLAPSED",
  value: { a!richTextIcon(icon: "shield") }
)
```

Error Prevention Checklist

Before submitting SAIL code, verify:

- [ ] **Text function usage**: Only use `text()` for type conversion, not string literals
- [ ] **Grid configuration:** Use `showSearchBox` only with Record Type data sources
- [ ] **Layout widths**:
- sideBySideLayout: `AUTO`, `MINIMIZE`, `1X`-`10X`

- formLayout: `FULL`, `WIDE`, `MEDIUM`, `NARROW`, `EXTRA_NARROW`
- [ ] **Component names:** Use exact component names from documentation
- [ ] **Parameter values**: Use only documented parameter values
- [ ] **Component parameters**: Include only documented parameters
- [ ] **Component availability**: Verify component exists in current Appian version
- [ ] **Layout nesting**: Check component compatibility within layouts
- [ ] **Required input indicators**: Do not add asterisks to required input labels since they are added automatically
- [ ] **Pie chart legend syntax**: Use `seriesLabelStyle: "LEGEND"` to show legend for pie charts
- [ ] **Inconsistent buttons styles for `a!fileUploadField`**: Use old button style values for `a!fileUploadField` only
- [ ] **Valid icons:** Check that icons are part of supported set before including